

EJERCICIO 3 DE REPASO DEL TEMA 1

Dada la clase Fecha y la clase Cuenta

```
class Fecha {
    int dia, mes, anio;
public:
    Fecha(int d, int m, int a) {
        dia=d; mes=m; anio=a;
    }
    int getDia() const { return dia; }
    int getMes() const { return mes; }
    int getAnio() const { return anio; }
};

class Cuenta {
    long int numCuenta;
    long int dni;
    char *nombre;
    Fecha fechaAlta;
    float tipoInteres;
    float tipoInteresMaximo;
public:
    ...
}
```

Implementa la clase Cuenta, teniendo en cuenta que:

- Queremos que el tipo de interés sea común a todas las cuentas y que dicho tipo nunca pueda ser superior al tipo de interés máximo (tipo de interés inicial = 1.25%, tipo interés máximo = 5%). El tipo de interés podrá variar a lo largo del tiempo, pero el tipo de interés máximo debe ser inamovible.
- Queremos que el número de cuenta sea generado automáticamente por la clase, de manera que cada cuenta creada tenga un número único (no repetido). La primera cuenta creada debe tener el número de cuenta 84934400 y el resto de cuentas números consecutivos a éste. Una vez creada una cuenta su número de cuenta no puede variar.
- Para dar de alta una cuenta será obligatorio facilitar la siguiente información: dni, nombre y fecha de alta. No es posible crear una cuenta sin facilitar estos datos.
- La clase Fecha proporcionada está completamente implementada (no se pueden añadir más métodos o funciones a dicha clase).
- La clase Cuenta debe proporcionar métodos públicos que permitan ver y modificar sus datos.
- La clase Cuenta debe permitir visualizar la información de la cuenta en cualquier dispositivo de salida (pantalla, impresora, fichero, etc.). El formato de salida debe ser:

numCuenta (tipoInteres en %) – nombre (dni) fechaAlta en formato dd/mm/aaaa

Ej: 84934403 (1.25%) – Juan (75678848) 07/03/2024

SOLUCION:

- Tipo de interés y tipo de interés máximo deben ser `static` ya que deben ser común a todas las cuentas.
- El tipo de interés máximo y el número de cuenta deben ser `constantes`.
- Para generar números consecutivos debemos usar un `contador static` (común a todas las cuentas) y como la 1ª cuenta creada debe tener como número de cuenta 84934400 vamos a crear una constante `static` (común a todas las cuentas) para guardar este dato.
- El constructor de la clase Cuenta debe tener obligatoriamente 3 parámetros (dni, nombre y fecha).
- Como la clase tiene memoria dinámica (atributo `char* nombre`), debemos programar un constructor de constructor de copia y un operador de asignación, ya que los que genera de oficio el compilador (hacen una copia binaria de los datos) no funcionarán correctamente. También debemos programar un destructor para liberar (delete) la memoria reservada de forma dinámica (new) en el constructor.

- DILEMA constructor de copia:

El constructor de copia de Cuenta es el que le indica al compilador como generar una copia de una cuenta. Cuando pasamos un objeto Cuenta por referencia como parámetro a una función o a un método no se crea una copia del objeto, pero cuando lo pasamos por valor (por copia) el compilador creará una copia del objeto siguiendo las instrucciones programadas en el constructor de copia.

Por tanto, debemos programar el constructor de copia de forma que devuelva una copia exacta del objeto. Si hacemos esto, en el `main()` no se debería permitir crear un objeto Cuenta a partir de otro objeto Cuenta (ya que entonces tendrían el mismo número de cuenta).

```
int main() {  
    Cuenta a(75444869, "Juan", Fecha(4,5,2024);  
    Cuenta b(a); //no se debe permitir  
}
```

Si queremos que en el `main()` se pueda crear una cuenta a partir de otra entonces NUNCA podremos pasar un objeto cuenta por valor (por copia) a una función o método (ya que entonces la copia tendrá un número de cuenta diferente al original) y SIEMPRE tendremos que pasar los objetos Cuenta por referencia (`const Cuenta &` para evitar que accidentalmente sea modificado).

Es responsabilidad del programador sopesar estas circunstancias y ser consciente de lo que implica tomar una decisión u otra.

Si en el enunciado del problema no se indica nada, debemos programar el constructor de copia de forma que haga una copia exacta, para poder pasar objetos por copia a las funciones o métodos.

- DILEMA operador de asignación:

Cuando un objeto es constante o parcialmente constante (tiene algunos atributos que son constantes) no tiene sentido permitir el uso del operador de asignación en el `main()`, ya que no se puede cambiar el objeto o parte del objeto y por tanto no se debería programar el `operator=` (el compilador de hecho, no lo genera cuando la clase tiene atributos que son constantes) ni siquiera cuando la clase tenga memoria dinámica (atributos tipo puntero).

Sólo lo programaremos si específicamente lo indican o si en el `main()` aparece alguna instrucción que lo usa, pero en ese caso, sólo podremos modificar los atributos que no son constantes.

Si en el enunciado del problema no se indica nada, no programaremos el operador de asignación cuando la clase tenga algunos atributos (que no sean `static`) constantes.

```
int main() {  
    Cuenta a(75444869, "Juan", Fecha(4,5,2024), b(75666641, "Luis", Fecha(12,12,2023);  
    a=b; //a tendrá el mismo nombre y fecha que b, pero su número de cuenta seguirá siendo el mismo  
}
```

CÓDIGO SOLUCION:

Fecha.h

```
#ifndef FECHA_H
#define FECHA_H

#include <iostream>

using namespace std;

class Fecha {
    int dia, mes, anio;
public:
    Fecha(int d, int m, int a) {
        dia=d; mes=m; anio=a;
    }
    //Fecha(const Fecha &f); //constructor de copia (LO GENERA EL COMPILADOR)
    //Fecha& operator=(const Fecha &f); //operador de asignacion (LO GENERA EL COMPILADOR)
    int getDia() const { return dia; }
    int getMes() const { return mes; }
    int getAnio() const { return anio; }
};

//ostream & operator<<(ostream &s, Fecha f);
```

Fecha.cpp

```
#include "Fecha.h"
/*
ostream & operator<<(ostream &s, Fecha f) {
    s << f.getDia() << "/" << f.getMes() << "/" << f.getAnio();
    return s;
} */
```

Cuenta.h

```
#ifndef CUENTA_H
#define CUENTA_H

#include "Fecha.h"

class Cuenta {
    const long int numCuenta;
    long int dni;
    char *nombre;
    Fecha fechaAlta;
    static float tipoInteres;
    static long int contador;
    static const long int numCuentaInicial;
    static const float tipoInteresMaximo;
public:
    Cuenta(long int d, char *nom, Fecha f);
    virtual ~Cuenta();
    Cuenta(const Cuenta& c);
    Cuenta& operator=(const Cuenta& c); //no lo programamos y ni siquiera escribimos la cabecera
    Cuenta& operator=(const Cuenta& c);

    long int getNumCuenta() const { return numCuenta; }
    long int getDni() const { return dni; }
    const char *getNombre() const { return nombre; } //acordaros de poner const al inicio
    Fecha getFechaAlta() const { return fechaAlta; }
    static float getTipoInteres() { return tipoInteres; }
    static long int getContador() { return contador; }
    static long int getNumCuentaInicial() { return numCuentaInicial; }
    static float getTipoInteresMaximo() { return tipoInteresMaximo; }

    void setDni(long int dni) { this->dni=dni; }
    void setNombre(char *nombre);
    void setFechaAlta(Fecha f) { fechaAlta=f; }
    static void setTipoInteres(float tipo);

    friend ostream & operator<<(ostream &s, const Cuenta &c);
};

ostream & operator<<(ostream &s, const Cuenta &c);
```

Cuenta.cpp

```
#include <iostream> //cout, cin
#include <cstring> //strlen, strcpy
#include "Cuenta.h"

using namespace std;

float Cuenta::tipoInteres=0.01;
long int Cuenta::contador=0;
const long int Cuenta::numCuentaInicial=84934400;
const float Cuenta::tipoInteresMaximo=0.05;

//fechaAlta(f.getDia(), f.getMes(), f.getAnio())
Cuenta::Cuenta(long int d, char *nom, Fecha f):numCuenta(numCuentaInicial+contador), fechaAlta(f) {
    dni=d;
    nombre=new char[strlen(nom)+1];
    strcpy(nombre, nom);
    contador++;
}

Cuenta::~Cuenta() {
    delete [] nombre;
}

Cuenta::Cuenta(const Cuenta& c):numCuenta(c.numCuenta), fechaAlta(c.fechaAlta) {
    dni=c.dni;
    nombre=new char[strlen(c.nombre)+1];
    strcpy(nombre, c.nombre);
}

Cuenta::Cuenta(const Cuenta& c):numCuenta(numCuentaInicial+contador), fechaAlta(c.fechaAlta) {
    dni=c.dni;
    nombre=new char[strlen(c.nombre)+1];
    strcpy(nombre, c.nombre);
    contador++;
}

Cuenta& Cuenta::operator=(const Cuenta& c) {
//NO LO PROGRAMAMOS (ELIMINAMOS TODO ESTO)
}

Cuenta& Cuenta::operator=(const Cuenta& c) {
    if (this != &c) {
        dni=c.dni;
        delete [] nombre;
        nombre=new char[strlen(c.nombre)+1];
        strcpy(nombre, c.nombre);
        fechaAlta=c.fechaAlta;
        //numCuenta no se puede cambiar porque es constante
    }
    return *this;
}

void Cuenta::setNombre(char *nombre) {
    delete [] this->nombre;
    this->nombre=new char[strlen(nombre)+1];
    strcpy(this->nombre, nombre);
}

void Cuenta::setTipoInteres(float tipo) {
    if (tipo>tipoInteresMaximo)
        tipo=tipoInteresMaximo;
    tipoInteres=tipo;
}

ostream & operator<<(ostream &s, const Cuenta &c) {
    s << c.numCuenta << " (" << c.tipoInteres*100 << "%" << " - " << c.nombre << " (" << c.dni << " ) "
    << c.fechaAlta.getDia() << "/" << c.fechaAlta.getMes() << "/" << c.fechaAlta.getAnio();
    return s;
}

/*
ostream & operator<<(ostream &s, const Cuenta &c) {
    s << c.numCuenta << " (" << c.tipoInteres*100 << "%" << " - " << c.nombre << " (" << c.dni << " ) "
    << c.fechaAlta;
    return s;
}
*/
```

main.cpp

```
#include <iostream>
#include "Cuenta.h"
#include "Fecha.h"

using namespace std;

int main() {
    cout << "Hay creadas " << Cuenta::getContador() << " cuentas" << endl;
    cout << "tipo de interes por defecto: " << Cuenta::getTipoInteres() << endl;
    Fecha f1(2,3,2024), f2(5,10,2024), f3(f2);
    // cout << f1 << endl << f2 << endl << f3 << endl;
    cout << "-----\n\n";
    Cuenta a(123456, "Pepe", f1);
    Cuenta b(126768, "Juanito", f2);

    cout << "Hay creadas " << Cuenta::getContador() << " cuentas" << endl;
    cout << a << endl << b << endl;

    cout << "Hay creadas " << Cuenta::getContador() << " cuentas" << endl;
    Cuenta::setTipoInteres(0.035);
    a.setDni(987654);
    b.setNombre("Luis");
    b.setFechaAlta(Fecha(1,1,2000));
    a.setFechaAlta(f2);
    cout << a.getNumCuenta() << endl << b.getNumCuenta() << endl;

    cout << a << endl << b << endl;
    cout << "Hay creadas " << Cuenta::getContador() << " cuentas" << endl;
    return 0;
}
```

```
Hay creadas 0 cuentas
tipo de interes por defecto: 0.01
-----
```

```
Hay creadas 2 cuentas
84934400 (1%) - Pepe (123456) 2/3/2024
84934401 (1%) - Juanito (126768) 5/10/2024
Hay creadas 2 cuentas
84934400
84934401
84934400 (3.5%) - Pepe (987654) 5/10/2024
84934401 (3.5%) - Luis (126768) 1/1/2000
Hay creadas 2 cuentas
```

main.cpp

```
#include <iostream>
#include "Cuenta.h"
#include "Fecha.h"

using namespace std;

int main() {
    cout << "Hay creadas " << Cuenta::getContador() << " cuent
    cout << "tipo de interes por defecto: " << Cuenta::getTipo
    Fecha f1(2,3,2024), f2(5,10,2024), f3(f2);
    // cout << f1 << endl << f2 << endl << f3 << endl;
    cout << "-----\n\n";
    Cuenta a(123456, "Pepe", f1);
    Cuenta b(126768, "Juanito", f2);
    Cuenta c(b);

    cout << "Hay creadas " << Cuenta::getContador() << " cuentas" << endl;
    cout << a << endl << b << endl << c << endl;
    a=b;

    cout << "Hay creadas " << Cuenta::getContador() << " cuentas" << endl;
    Cuenta::setTipoInteres(0.035);
    a.setDni(987654);
    b.setNombre("Luis");
    b.setFechaAlta(Fecha(1,1,2000));
    a.setFechaAlta(f2);
    cout << a.getNumCuenta() << endl << b.getNumCuenta() << endl;

    cout << a << endl << b << endl << c << endl;
    cout << "Hay creadas " << Cuenta::getContador() << " cuentas" << endl;
    return 0;
}
```

```
Hay creadas 0 cuentas
tipo de interes por defecto: 0.01
-----
```

```
Hay creadas 3 cuentas
84934400 (1%) - Pepe (123456) 2/3/2024
84934401 (1%) - Juanito (126768) 5/10/2024
84934402 (1%) - Juanito (126768) 5/10/2024
Hay creadas 3 cuentas
84934400
84934401
84934400 (3.5%) - Juanito (987654) 5/10/2024
84934401 (3.5%) - Luis (126768) 1/1/2000
84934402 (3.5%) - Juanito (126768) 5/10/2024
Hay creadas 3 cuentas
```

Ejecuta el **main.cpp** eliminado el `const` y el `&` en la función `operator<<` (.h y .cpp) y vea lo que ocurre

```
ostream & operator<<(ostream &s, Cuenta c)
```

¿entiendes por qué?

```
Hay creadas 0 cuentas
tipo de interes por defecto: 0.01
-----
```

```
Hay creadas 3 cuentas
84934403 (1%) - Pepe (123456) 2/3/2024
84934404 (1%) - Juanito (126768) 5/10/2024
84934405 (1%) - Juanito (126768) 5/10/2024
Hay creadas 6 cuentas
84934400
84934401
84934406 (3.5%) - Juanito (987654) 5/10/2024
84934407 (3.5%) - Luis (126768) 1/1/2000
84934408 (3.5%) - Juanito (126768) 5/10/2024
Hay creadas 9 cuentas
```